	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
UNIVERSIDAD DON BOSCO	MATERIA	DESARROLLO DE SOFTWARE EMPRESARIAL	GUIA DE LABORATORIO #5
	PRÁCTICA	API EN .NET CON ARQUITECTURA N-CAPAS	

I. OBJETIVOS

- Que el estudiante aprenda a identificar cada una de las capas y su responsabilidad.
- Que el estudiante aprenda a crear APIs en .NET Core.
- Que el estudiante aprenda sobre el uso de swagger.
- Que el estudiante aprenda sobre el uso de dapper y automapper.

II. INTRODUCCIÓN TEORICA

Arquitectura N-Capas

La arquitectura en capas es un modelo de diseño de software, cuya base es la separación de las diferentes funcionalidades del sistema en capas o niveles, donde cada capa se encarga de un conjunto de tareas específicas y se comunica con los niveles adyacentes mediante interfaces bien definidas.

Cada capa se construye sobre la inferior inmediata, la cual proporciona los servicios y la funcionalidad necesarios para que la capa superior pueda funcionar correctamente. Este modelo permite una mejor organización y modularidad del sistema; además facilita su mantenimiento y evolución a largo plazo.

Capas Comunes en la Arquitectura N-Capas

1. Capa de Presentación (Presentation Layer)
   - **Responsabilidad:** Manejar la interfaz de usuario y la interacción.
   - **Tecnologías comunes:** HTML, CSS, JavaScript, Angular, React, Blazor.
   - **Ejemplo:** Formulario web para ingresar datos de un cliente.

2. Capa de Aplicación (Application Layer)
   - **Responsabilidad**: Contiene la lógica de negocio o reglas de la aplicación.
   - **Tecnologías comunes**: .NET Core, Java EE, Spring.
   - **Ejemplo**: Validación de datos ingresados por el usuario.

3. Capa de Negocio (Business Logic Layer)
   - **Responsabilidad**: Implementar las reglas de negocio y lógica empresarial.
   - **Tecnologías comunes**: Servicios de negocio, componentes de dominio, microservicios.
   - **Ejemplo**: Cálculo de precios, procesamiento de órdenes.

4. Capa de Acceso a Datos (Data Access Layer)
   - **Responsabilidad**: Interactuar con la base de datos u otros sistemas de almacenamiento de datos.
   - **Tecnologías comunes**: Entity Framework, Hibernate, ADO.NET.
   - **Ejemplo**: Consultas SQL para recuperar datos del cliente.

5. Entidades de negocio (Business entities)
   - **Responsabilidad**: Representar los objetos del dominio del negocio, contener la estructura de datos que se utilizará a lo largo de la aplicación.
   - **Tecnologías comunes**: Clases POCO/POJO en C#, Java, etc.
   - **Ejemplo**: Clase Customer con propiedades Id, Name, Email que representa a un cliente en el sistema.



Ilustración 1 - Diagrama de interacción de un modelo de N-Capas

Existen varias arquitecturas de capas utilizadas en el desarrollo de software. La diferencia entre ellas radica en cómo se organizan las responsabilidades y la comunicación entre los componentes. Algunas de las más utilizadas son la Arquitectura Hexagonal (también conocida como Arquitectura de Puertos y Adaptadores) y la Arquitectura Limpia. Es posible implementar una arquitectura de capas propio siempre y cuando tenga una justificación técnica y una responsabilidad claramente definida por cada capa.

Swagger

Swagger es un marco de software que ayuda a los desarrolladores a diseñar, construir, documentar y consumir servicios web RESTful. Swagger proporciona una interfaz de usuario interactiva para explorar y probar las APIs.

Componentes de Swagger

- **Swagger UI**: es una interfaz de usuario interactiva que permite explorar y probar las APIs documentadas. Permite a los desarrolladores y consumidores de la API ver la documentación generada y probar los endpoints directamente desde el navegador.

- **Swagger Editor**: es un editor en línea que permite crear y modificar archivos de especificación OpenAPI. Facilita la creación y edición de descripciones de API en formato YAML o JSON.

- **Swagger Codegen**: es una herramienta que genera código cliente y servidor a partir de la especificación OpenAPI. Acelera el desarrollo al generar stubs de código en varios lenguajes de programación.

- **SwaggerHub**: Una plataforma colaborativa para diseñar y documentar APIs. Permite la colaboración en tiempo real en el diseño y documentación de APIs.

OpenAPI

Es un estándar para describir APIs RES**T** de forma estructurada y legible tanto para humanos como para herramientas. Permite definir cómo funciona una API: qué rutas existen, qué datos recibe, qué respuestas devuelve, qué parámetros utiliza y cómo se autentica.

Dapper

Dapper es un micro ORM (Object-Relational Mapper) para .NET que proporciona una forma sencilla y rápida de interactuar con bases de datos. A diferencia de los ORM completos como Entity Framework, Dapper es ligero y de alto rendimiento, centrándose en la ejecución de consultas SQL y el mapeo de resultados a objetos de .NET.

Características

1. Ligero y Rápido:
   - Minimalista y diseñado para ser rápido, con baja sobrecarga.
   - Ideal para aplicaciones que requieren un rendimiento máximo.

2. Soporte para SQL Nativo:
   - Permite escribir y ejecutar consultas SQL nativas.
   - Ofrece flexibilidad total en la construcción de consultas.

3. Mapeo Automático:
   - Mapea automáticamente los resultados de las consultas SQL a objetos de .NET.
   - Admite tipos de datos complejos y relaciones de entidades.

4. Compatibilidad:
   - Compatible con cualquier base de datos soportada por ADO.NET.
   - Funciona bien junto con otros ORM o frameworks de acceso a datos.

AutoMapper

AutoMapper es una biblioteca para .NET diseñada para simplificar el mapeo de objetos de un tipo a otro. Facilita la conversión entre diferentes modelos, eliminando la necesidad de escribir código manual para copiar propiedades entre objetos.

Características de AutoMapper

1. Configuración Basada en Convenciones:
   - Mapea automáticamente propiedades con nombres coincidentes entre los tipos origen y destino.
   - Permite configuraciones personalizadas cuando las convenciones no son suficientes.

2. Soporte para Proyecciones:
   - Convierte colecciones y listas de objetos.
   - Admite la proyección de consultas LINQ directamente en los tipos destino.

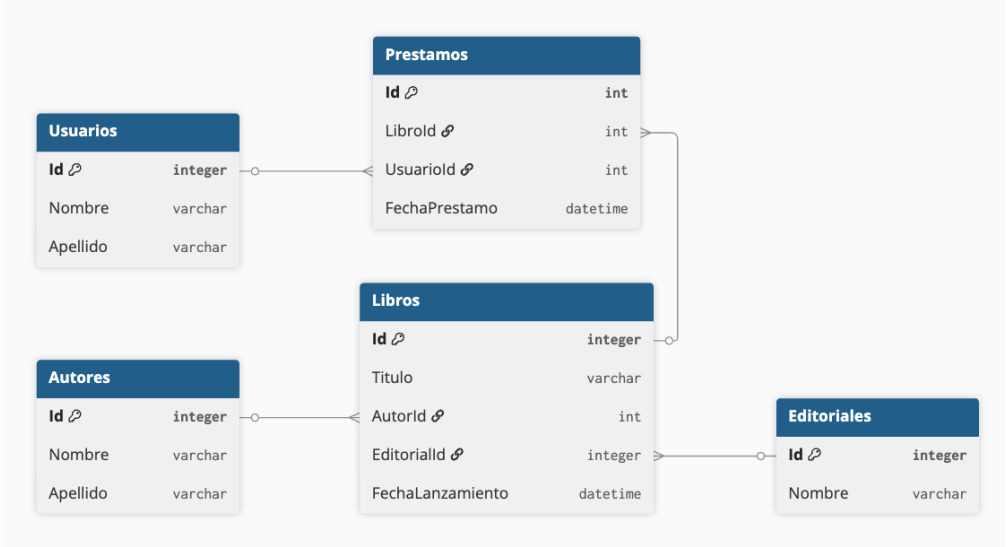
3. Extensible:
   - Permite crear mapeos personalizados y resolver conflictos de mapeo.
   - Soporta resolvers y convertidores personalizados.

4. Optimización:
   - Mejora el rendimiento de la aplicación mediante mapeos preconfigurados y reutilizables.

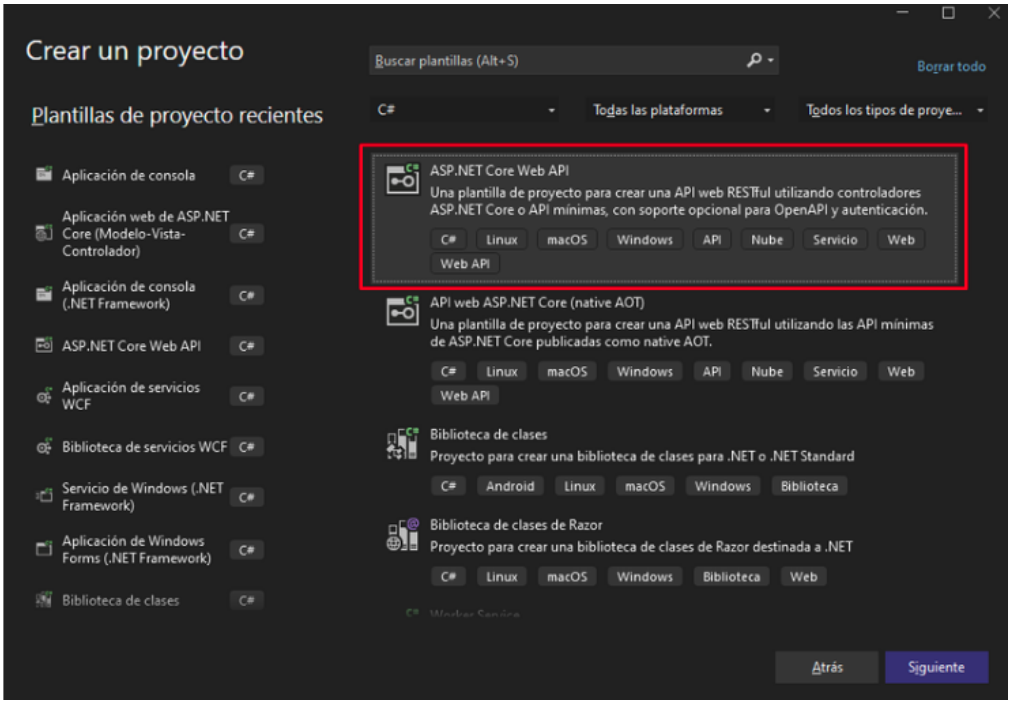
I. DESARROLLO DE LA PRÁCTICA

Parte 1 – Creación de APIs con arquitectura de N-Capas

1. En SQL Server, cree una base de datos llamada **Biblioteca**, utilice el asistente o realice los scripts necesarios para crear las siguientes tablas.



2. Cree un nuevo proyecto en Visual Studio 2026 con el nombre **Biblioteca.API**, asegúrese de elegir la plantilla **ASP.NET Core Web API**.



3. Observe que ha creado un nuevo proyecto, el cual tiene implementado un controlador de ejemplo con el nombre **WeatherForecastController**.

```
using Microsoft.AspNetCore.Mvc;

namespace Biblioteca.API.Controllers;

[ApiController]
[Route( template: "[controller]")]
public class WeatherForecastController : ControllerBase  2 usages
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };

    private readonly ILogger<WeatherForecastController> _logger;

    public WeatherForecastController(ILogger<WeatherForecastController> logger)
    {
        _logger = logger;
    }

    [HttpGet(Name = "GetWeatherForecast")]
    public IEnumerable<WeatherForecast> Get()
    {
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
            {
                Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                TemperatureC = Random.Shared.Next(-20, 55),
                Summary = Summaries[Random.Shared.Next(Summaries.Length)]
            }) // IEnumerable<WeatherForecast>
            .ToArray(); // WeatherForecast[]
    }
}
```

4. En el Administrador de Paquetes NuGet, busque e instale el paquete **Swashbuckle.AspNetCore**, este paquete servirá para utilizar la **Swagger UI**. Asegúrese de seleccionar una versión del mismo nivel de versión que el paquete **Microsoft.AspNetCore.OpenApi**.

5. Modifique la clase **Program.cs** como se muestra a continuación.

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

// Swagger UI
builder.Services.AddSwaggerGen(); // ← NUEVO

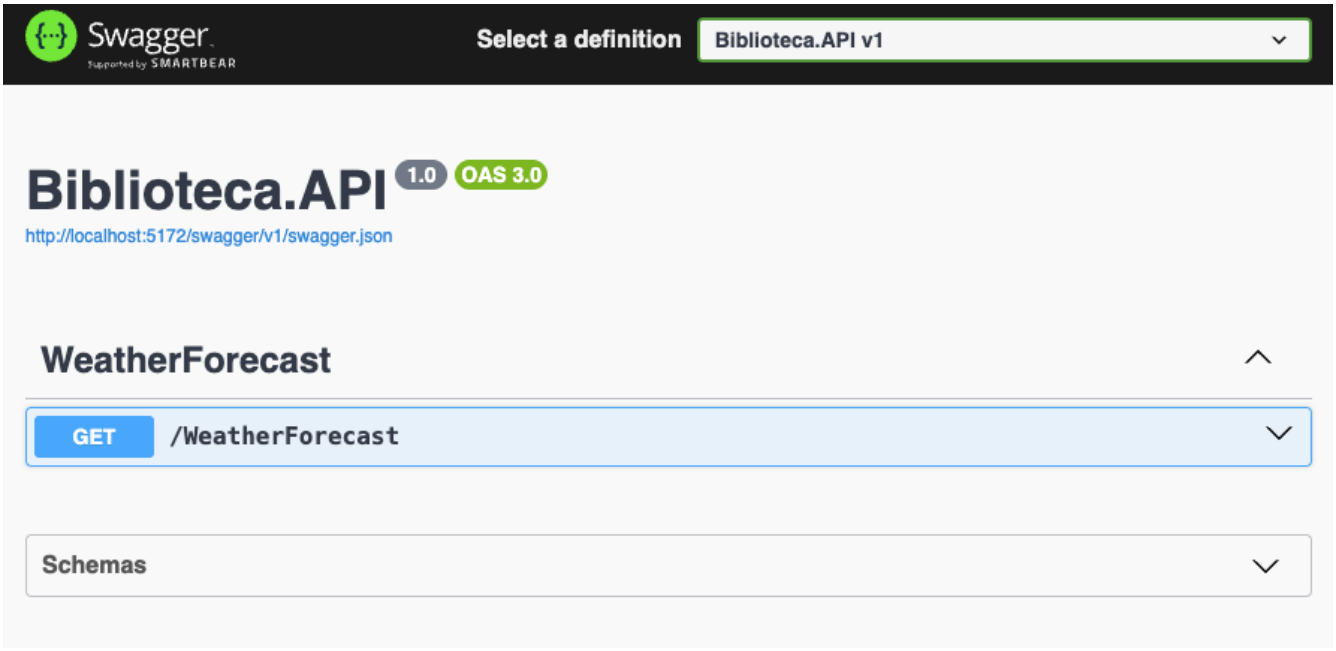
var app :WebApplication = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();

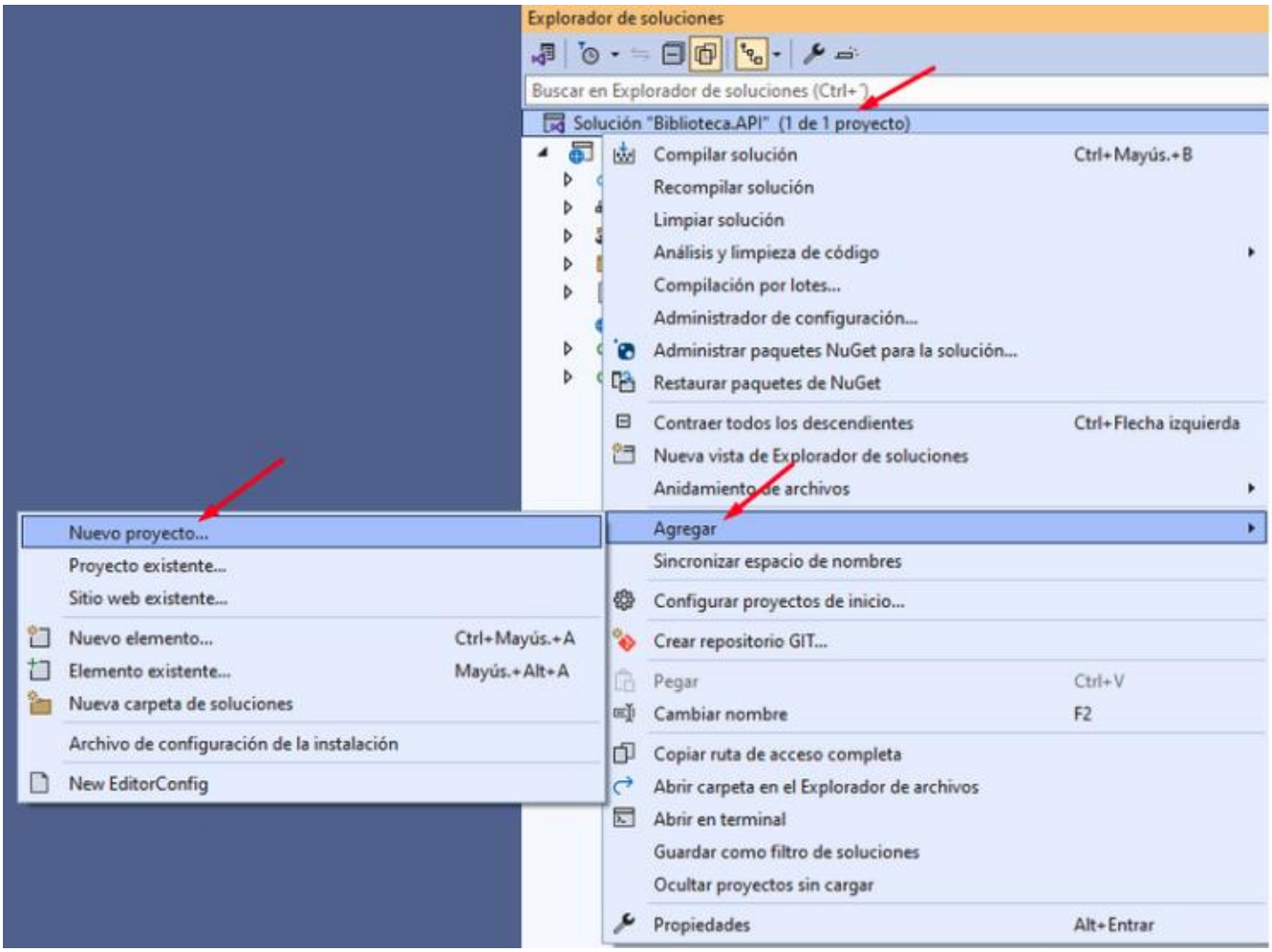
    // Interfaz Swagger UI
    app.UseSwagger(); // ← NUEVO
    app.UseSwaggerUI(); // ← NUEVO
}
```

- Swagger se habilita únicamente en el entorno de desarrollo.

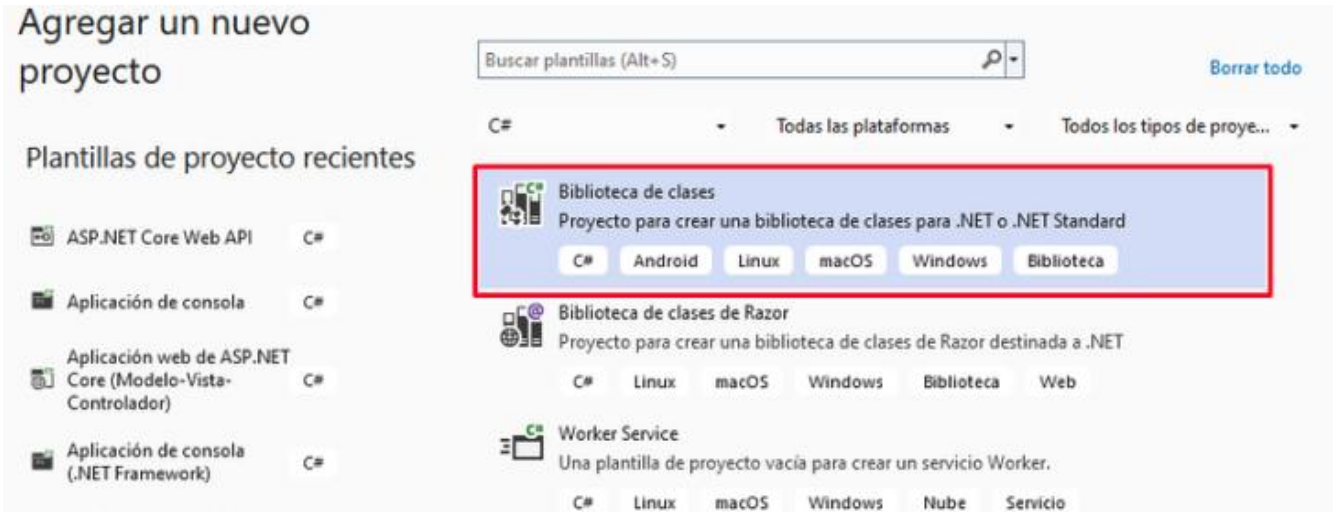
6. Ejecute el proyecto y en el navegador de su preferencia ingrese la URL https://localhost:xxxx/swagger, asegúrese de sustituir **"xxxx"** por el puerto asignado a su aplicación.



6. Sobre el Explorador de soluciones, haga clic derecho y seleccione la opción **Biblioteca.API > Agregar > Nuevo proyecto**.



7. Seleccione la plantilla **Biblioteca de clases** y haga clic en **Siguiente**.



8. Asigne el nombre **Biblioteca.BL** al proyecto.

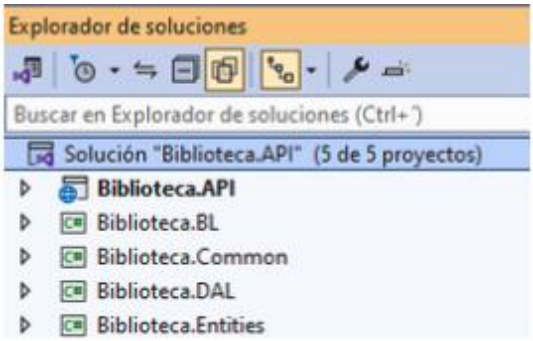


9. Seleccione el mismo Framework con el cual creo el proyecto **Biblioteca.API** y haga clic en **Crear**.

10. De la misma manera cree los siguientes proyectos de tipo **Biblioteca de clases**.

- Biblioteca.Common
- Biblioteca.DAL
- Biblioteca.Entites

11. Una vez finalizada la creación de los proyectos, la estructura de la solución debe verse como se muestra a continuación.



Parte 2 – Configuración del proyecto Biblioteca.Common

1. Agregue una nueva clase con el nombre **AppSettings** en el proyecto **Biblioteca.Common.** Modifique el contenido de la clase con el siguiente código**.**

```
namespace Biblioteca.Common;

public class AppSettings
{
    public string ConnectionString { get; set; } = string.Empty;
}
```

- El proyecto **Biblioteca.Common** no forma parte de una capa especifica dentro de la arquitectura, más bien, se utiliza para compartir funcionalidad entre las múltiples capas.

2. Ubique el archivo **appsettings.json** del proyecto **Biblioteca.API** y agregue una nueva sección con el nombre **AppSettings**, luego agregue la cadena de conexión de la siguiente forma.

```
{
  "AppSettings": {
    "ConnectionString": "Server=(localdb)\\mssqllocaldb;Database=Biblioteca;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

3. Modifique la clase **Program.cs** del proyecto **Biblioteca.API** y agregue el siguiente código.

```
using Biblioteca.Common;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

// Swagger UI
builder.Services.AddSwaggerGen();

builder.Services.Configure<AppSettings>(builder.Configuration.GetSection(key: "AppSettings")); // ← NUEVO

var app :WebApplication = builder.Build();
```

Parte 3 – Configuración del proyecto Biblioteca.Entities

1. En el proyecto **Biblioteca.Entities** agregue un nuevo directorio con el nombre **Models** y **Dtos**.

2. Dentro de la carpeta **Models** cree una nueva clase **Autor** y agregue el siguiente código**.**

```
using System.ComponentModel.DataAnnotations;


namespace Biblioteca.Entites.Models;
```

```
public class Autor
{
    [Key]
    public int Id { get; set; }

    public required string Nombre { get; set; }

    public required string Apellido { get; set; }
}
```

3. Dentro de la carpeta **Dtos** cree la clase **AutorDto** y modifique el código como se muestra a continuación.

```
using System.ComponentModel.DataAnnotations;

namespace Biblioteca.Entites.Dtos;

public class AutorDto
{
    public int Codigo { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio")]
    [StringLength(50, ErrorMessage = "El campo {0} no puede tener mas de {1} caracteres")]
    public string Nombre { get; set; } = string.Empty;

    [Required(ErrorMessage = "El campo {0} es obligatorio")]
    [StringLength(50, ErrorMessage = "El campo {0} no puede tener mas de {1} caracteres")]
    public string Apellido { get; set; } = string.Empty;
}
```

Hasta el momento se han creado dos clases que representa a la misma tabla **Autor**, pero cada una tiene su propósito. En el desarrollo de software, el modelo se refiere a la representación de las entidades de dominio y su lógica asociada, típicamente vinculadas a la base de datos y con estructuras más complejas que pueden incluir validaciones y métodos específicos.

Los **DTOs (Data Transfer Objects)** son estructuras simplificadas utilizadas para transferir datos entre diferentes capas de la aplicación, diseñados para mejorar la eficiencia y la claridad en la comunicación sin exponer la complejidad interna de los modelos de dominio.

Para mantener una separación adecuada entre las capas de la aplicación y evitar exponer directamente las entidades internas, se utilizan DTOs (Data Transfer Objects) como modelos de intercambio de información. Los DTOs permiten controlar los datos que se envían y reciben mediante la API, desacoplando la estructura interna del sistema del contrato externo.

Esta separación puede implicar que las propiedades de las entidades y los DTOs tengan nombres o estructuras diferentes, por lo que es necesario realizar un proceso de mapeo entre ambas clases. Por ejemplo, la propiedad **Id** de la entidad **Autor** puede corresponder a la propiedad **Código** del DTO. Realizar estos mapeos manualmente puede resultar repetitivo y propenso a errores cuando existen muchas propiedades.

Para simplificar este proceso se utiliza AutoMapper, una biblioteca que permite configurar y automatizar la conversión entre objetos, reduciendo la cantidad de código necesario para realizar estos mapeos.

Parte 4 – Configuración del proyecto Biblioteca.DAL

1. Sobre el proyecto **Biblioteca.DAL** instale los siguientes paquetes:

- Dapper
- Microsoft.Extensions.Options
- Microsoft.Data.SqlClient

2. Cree dos nuevas carpetas, una con el nombre **Interfaces** y la otra **Repositories** en el proyecto **Biblioteca.DAL.**

3. Cree una nueva **Interfaz** con el nombre **IDatabaseRepository** en la carpeta **Interfaces** y modifíquela con el siguiente código.

```
using Dapper;

namespace Biblioteca.DAL.Interfaces;

public interface IDatabaseRepository
{
    public Task<List<T>> GetDataByQueryAsync<T>(string query, DynamicParameters? parameters = null);
    public Task<int> InsertAsync(string query, DynamicParameters? parameters = null);
    public Task<T?> UpdateAsync<T>(string query, DynamicParameters? parameters = null);
    public Task<bool> DeleteAsync(string query, DynamicParameters? parameters = null);
}
```

4. Cree una nueva clase **DatabaseRepository** en la carpeta **Repositories** del proyecto **Biblioteca.DAL** y modifíquela con el siguiente código. Asegúrese que la clase implemente la interfaz **IDatabaseRepository**.

```
using Biblioteca.Common;
using Biblioteca.DAL.Interfaces;
using Dapper;
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Options;

namespace Biblioteca.DAL.Repositories;

public class DatabaseRepository(IOptions<AppSettings> appSettings) : IDatabaseRepository
{
```

```
private readonly string _connectionString = appSettings.Value.ConnectionString;

public async Task<List<T>> GetDataByQueryAsync<T>(string query, DynamicParameters? parameters = null)
{
    try
    {
        await using var connection = new SqlConnection(_connectionString);
        connection.Open();
        var result = await connection.QueryAsync<T>(query, parameters);
        connection.Close();
        return result.ToList();
    }
    catch (Exception e)
    {
        throw new Exception("Error en la consulta GetDataByQueryAsync: " + e.Message);
    }
}

public async Task<int> InsertAsync(string query, DynamicParameters? parameters = null)
{
    try
    {
        await using var connection = new SqlConnection(_connectionString);
        connection.Open();
        var result = await connection.QuerySingleOrDefaultAsync<int>(query, parameters);
        connection.Close();
        return result;
    }
    catch (Exception e)
    {
        throw new Exception("Error en la consulta InsertAsync: " + e.Message);
    }
}

public async Task<T?> UpdateAsync<T>(string query, DynamicParameters? parameters = null)
{
    try
    {
        await using var connection = new SqlConnection(_connectionString);
        connection.Open();
        var result = await connection.QueryAsync<T>(query, parameters);
```

```
            connection.Close();

            var enumerable = result.ToList();

            if (enumerable.Any())
            {
                return enumerable.FirstOrDefault();
            }
        }
        catch (Exception e)
        {
            throw new Exception("Error en la consulta UpdateAsync: " + e.Message);
        }

        return default;
    }

    public async Task<bool> DeleteAsync(string query, DynamicParameters? parameters = null)
    {
        try
        {
            await using var connection = new SqlConnection(_connectionString);
            connection.Open();
            var result = await connection.QuerySingleOrDefaultAsync<bool>(query, parameters);
            connection.Close();
            return result;
        }
        catch (Exception e)
        {
            throw new Exception("Error en la consulta DeleteAsync: " + e.Message);
        }
    }
}
```

5. En la carpeta **Interfaces** del proyecto **Biblioteca.DAL**, agregue una nueva interfaz con el nombre **IAutorRepository** y agregue el siguiente código.

```
using Biblioteca.Entites.Models;

namespace Biblioteca.DAL.Interfaces;

public interface IAutorRepository
```

```
{
    public Task<List<Autor>> GetAutoresAsync();
    public Task<Autor?> GetAutorByIdAsync(int id);
    public Task<int> InsertAutorAsync(Autor autor);
    public Task<Autor?> UpdateAutorAsync(Autor autor);
    public Task<bool> DeleteAutorAsync(int id);
}
```

6. Cree la clase **AutorRepository** en la carpeta **Repositories** del proyecto Biblioteca.DAL y modifique el código como se muestra a continuación. Asegúrese que la clase implemente la interfaz **IAutorRepository**.

```
using Biblioteca.DAL.Interfaces;
using Biblioteca.Entites.Models;
using Dapper;

namespace Biblioteca.DAL.Repositories;

public class AutorRepository(IDatabaseRepository databaseRepository) : IAutorRepository
{
    public async Task<List<Autor>> GetAutoresAsync()
    {
        var query = "SELECT * FROM Autores";
        return await databaseRepository.GetDataByQueryAsync<Autor>(query);
    }

    public async Task<Autor?> GetAutorByIdAsync(int id)
    {
        var query = "SELECT * FROM Autores WHERE Id = @id";

        var parameters = new DynamicParameters();
        parameters.Add("@Id", id);

        return (await databaseRepository.GetDataByQueryAsync<Autor>(query, parameters)).FirstOrDefault();
    }

    public async Task<int> InsertAutorAsync(Autor autor)
    {
        var query = "INSERT INTO Autores (Nombre, Apellido) VALUES (@Nombre, @Apellido); SELECT
SCOPE_IDENTITY()";

        var parameters = new DynamicParameters();
        parameters.Add("@Nombre", autor.Nombre);
```

```
        parameters.Add("@Apellido", autor.Apellido);

        return await databaseRepository.InsertAsync(query, parameters);
    }

    public async Task<Autor?> UpdateAutorAsync(Autor autor)
    {
        var query = "UPDATE Autores SET Nombre = @Nombre, Apellido = @Apellido WHERE Id = @Id";
        var parameters = new DynamicParameters();
        parameters.Add("@Id", autor.Id);
        parameters.Add("@Nombre", autor.Nombre);
        parameters.Add("@Apellido", autor.Apellido);
        await databaseRepository.UpdateAsync<Autor>(query, parameters);
        return autor;
    }
    public async Task<bool> DeleteAutorAsync(int id)
    {
        var query = "DELETE FROM Autores WHERE Id = @Id";

        var parameters = new DynamicParameters();
        parameters.Add("@Id", id);

        return await databaseRepository.DeleteAsync(query, parameters);
    }
}
```

7. Cree un nuevo directorio **Services** en el proyecto **Biblioteca.DAL**, posteriormente, agregue una clase con el nombre **ServiceCollectionExtensions** y modifique el código de la siguiente manera.

```
using Biblioteca.DAL.Interfaces;
using Biblioteca.DAL.Repositories;
using Microsoft.Extensions.DependencyInjection;

namespace Biblioteca.DAL.Services;

public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddRepositoryConnector(this IServiceCollection services)
    {
        services.AddTransient<IDatabaseRepository, DatabaseRepository>();
        services.AddTransient<IAutorRepository, AutorRepository>();
        return services;
```

```
    }
}
```

- Esta clase facilita la inyección de dependencias, lo que reduce el acoplamiento entre clases y delega la responsabilidad de crear nuevas instancias a elementos externos a las clases.

Parte 5 – Configuración del proyecto Biblioteca.BL

1. En el proyecto **Biblioteca.BL** agregue tres nuevos directorios con los nombres **AutoMapper**, **Interfaces** y **Services**.

2. Instale el paquete NuGet **AutoMapper** en su última versión en el proyecto **Biblioteca.BL**.

3. En el directorio **Interfaces** del proyecto **Biblioteca.BL** agregue una nueva interfaz **IAutorService**, modifique el código de la siguiente manera.

```
using Biblioteca.Entites.Dtos;

namespace Biblioteca.BL.Interfaces;

public interface IAutorService
{
    public Task<List<AutorDto>> GetAllAutoresAsync();
    public Task<AutorDto?> GetAutorByIdAsync(int id);
    public Task<int> InsertAutorAsync(AutorDto autorDto);
    public Task<AutorDto?> UpdateAutorAsync(AutorDto autorDto);
    public Task<bool> DeleteAutorAsync(int id);
}
```

4. Cree una nueva clase **AutorService** en el directorio Services del proyecto **Biblioteca.BL** con el siguiente código.

```
using AutoMapper;
using Biblioteca.BL.Interfaces;
using Biblioteca.DAL.Interfaces;
using Biblioteca.Entites.Dtos;
using Biblioteca.Entites.Models;

namespace Biblioteca.BL.Services;

public class AutorService(
    IAutorRepository repository,
    IMapper mapper
    ) : IAutorService
{
```

```
public async Task<List<AutorDto>> GetAllAutoresAsync()
{
    try
    {
        var result = await repository.GetAutoresAsync();
        return mapper.Map<List<Autor>, List <AutorDto>> (result);
    }
    catch (Exception e)
    {
        return new List<AutorDto>();
    }
}

public async Task<AutorDto?> GetAutorByIdAsync(int id)
{
    try
    {
        var result = await repository.GetAutorByIdAsync(id);

        if (result == null)
            return null;

        return mapper.Map<Autor, AutorDto>(result);
    }
    catch (Exception e)
    {
        return null;
    }
}

public async Task<int> InsertAutorAsync(AutorDto autorDto)
{
    try
    {
        var entity = mapper.Map<AutorDto, Autor>(autorDto);
        return await repository.InsertAutorAsync(entity);
    }
    catch (Exception e)
    {
        return -1;
    }
```

```
    }

    public async Task<AutorDto?> UpdateAutorAsync(AutorDto autorDto)
    {
        try
        {
            var entity = mapper.Map<AutorDto, Autor>(autorDto);
            var result = await repository.UpdateAutorAsync(entity);

            if (result == null)
                return null;

            return mapper.Map<Autor, AutorDto>(result);
        }
        catch (Exception e)
        {
            return null;
        }
    }

    public async Task<bool> DeleteAutorAsync(int id)
    {
        try
        {
            return await repository.DeleteAutorAsync(id);
        }
        catch (Exception e)
        {
            return false;
        }
    }
}
```

5. Dentro de la carpeta **AutoMapper** agregue la clase **AutoMapperProfile.**

```
using AutoMapper;
using Biblioteca.Entites.Dtos;
using Biblioteca.Entites.Models;

namespace Biblioteca.BL.AutoMapper;
```

```
public class AutoMapperProfile : Profile
{
    public AutoMapperProfile()
    {
        CreateMap<Autor, AutorDto>()
            .ForMember(destination => destination.Codigo, opts => opts.MapFrom(source => source.Id))
            .ForMember(destination => destination.Nombre, opts => opts.MapFrom(source => source.Nombre))
            .ForMember(destination => destination.Apellido, opts => opts.MapFrom(source => source.Apellido))
            .ReverseMap();
    }
}
```

6. En la carpeta Services agregue una nueva clase **ServiceCollectionExtensions** con el siguiente código.

```
using Biblioteca.BL.AutoMapper;
using Biblioteca.BL.Interfaces;
using Biblioteca.DAL.Services;
using Microsoft.Extensions.DependencyInjection;

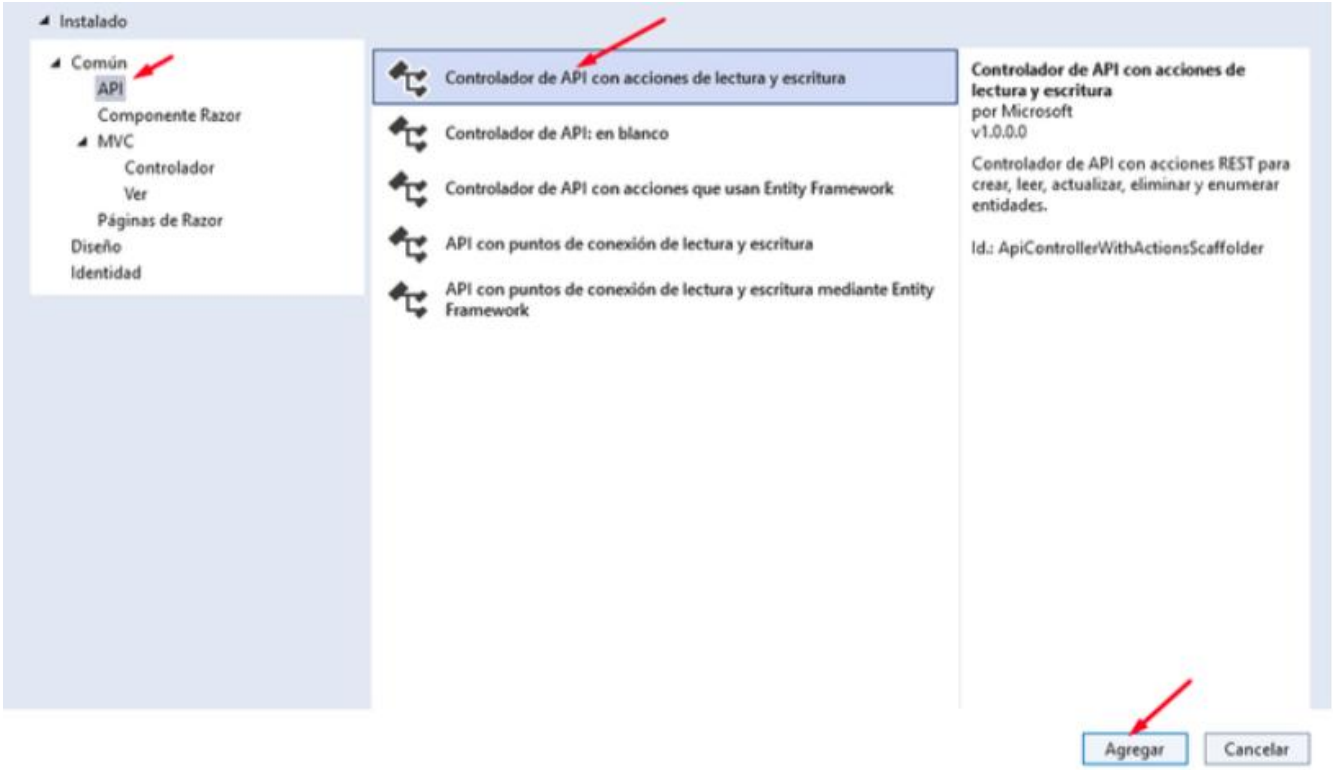
namespace Biblioteca.BL.Services;

public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddServiceConnector(this IServiceCollection services)
    {
        services.AddAutoMapper(_ => { }, typeof(AutoMapperProfile));
        services.AddTransient<IAutorService, AutorService>();
        services.AddRepositoryConnector();
        return services;
    }
}
```

Parte 6 – Configuración del proyecto Biblioteca.API

1. Sobre el directorio **Controllers** del proyecto Biblioteca.API haga clic derecho y selección la opción **Agregar > Controlador**.

2. Del cuadro de dialogo seleccione la opcion **API** y luego **Controlador API con acciones de lectura y escritura**.



3. Asigne el nombre **AutorController** y haga clic en **Agregar**.

4. Agregue el siguiente código en el controlador creado.

```
using Biblioteca.BL.Interfaces;
using Biblioteca.Entites.Dtos;
using Microsoft.AspNetCore.Mvc;

namespace Biblioteca.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AutorController(IAutorService service) : ControllerBase
    {
        // GET: api/<AutorController>
        [HttpGet]
        [ProducesResponseType(typeof(IEnumerable<IEnumerable<AutorDto>>), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status204NoContent)]
        public async Task<IActionResult> Get()
        {
            IEnumerable<AutorDto> result = await service.GetAllAutoresAsync();
            return result.Any() ? Ok(result) : NoContent();
        }
```

```
        // GET api/<AutorController>/5
        [HttpGet("{id}")]
        [ProducesResponseType(typeof(AutorDto), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status204NoContent)]
        public async Task<IActionResult> Get(int id)
        {
            var result = await service.GetAutorByIdAsync(id);
            return result != null ? Ok(result) : NoContent();
        }

        // POST api/<AutorController>
        [HttpPost]
        [ProducesResponseType(typeof(int), StatusCodes.Status201Created)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        public async Task<IActionResult> Post(AutorDto? dto)
        {
            if (dto == null)
            {
                return BadRequest();
            }

            var result = await service.InsertAutorAsync(dto);
            return result > 0 ? CreatedAtAction("Post", result) : BadRequest();
        }

        // PUT api/<AutorController>/5
        [HttpPut("{id}")]
        [ProducesResponseType(typeof(int), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        public async Task<IActionResult> Put(int id, AutorDto? dto)
        {
            if (dto == null)
            {
                return BadRequest();
            }

            var result = await service.UpdateAutorAsync(dto);
            return result != null ? CreatedAtAction("Post", result) : BadRequest();
        }

        // DELETE api/<AutorController>/5
```

```
        [HttpDelete("{id}")]
        [ProducesResponseType(typeof(int), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        public async Task<IActionResult> Delete(int id)
        {
            var result = await service.DeleteAutorAsync(id);
            return result ? Ok(result) : BadRequest();
        }
    }
}
```

5. Modifique el archivo **Program.cs** para agregue los servicios de inyección de dependencias de la capa BL y DL.

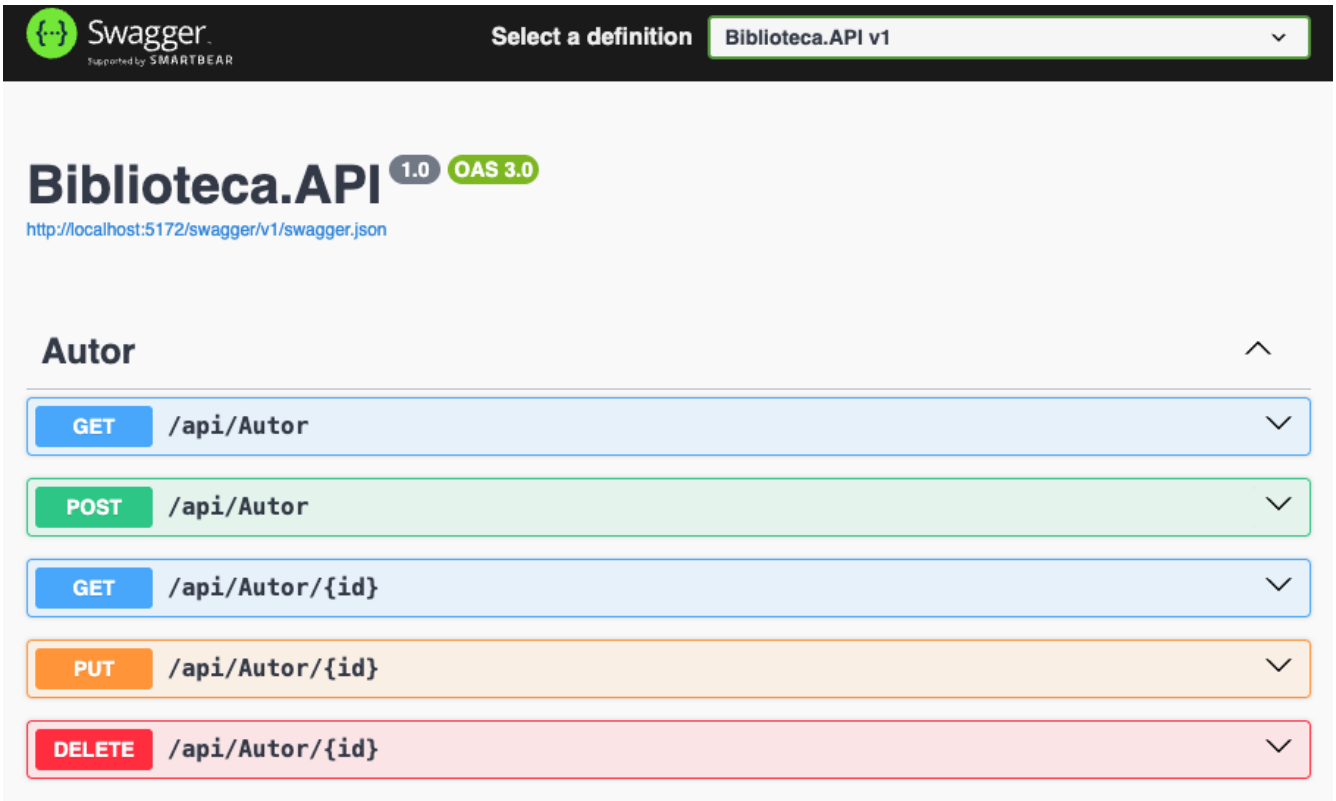
```
// Swagger UI
builder.Services.AddSwaggerGen();

builder.Services.Configure<AppSettings>(builder.Configuration.GetSection(key: "AppSettings"));

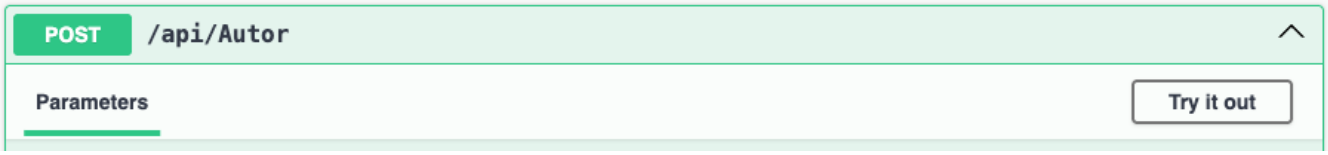
builder.Services.AddServiceConnector(); // ← NUEVO
builder.Services.AddRepositoryConnector(); // ← NUEVO

var app :WebApplication = builder.Build();
```

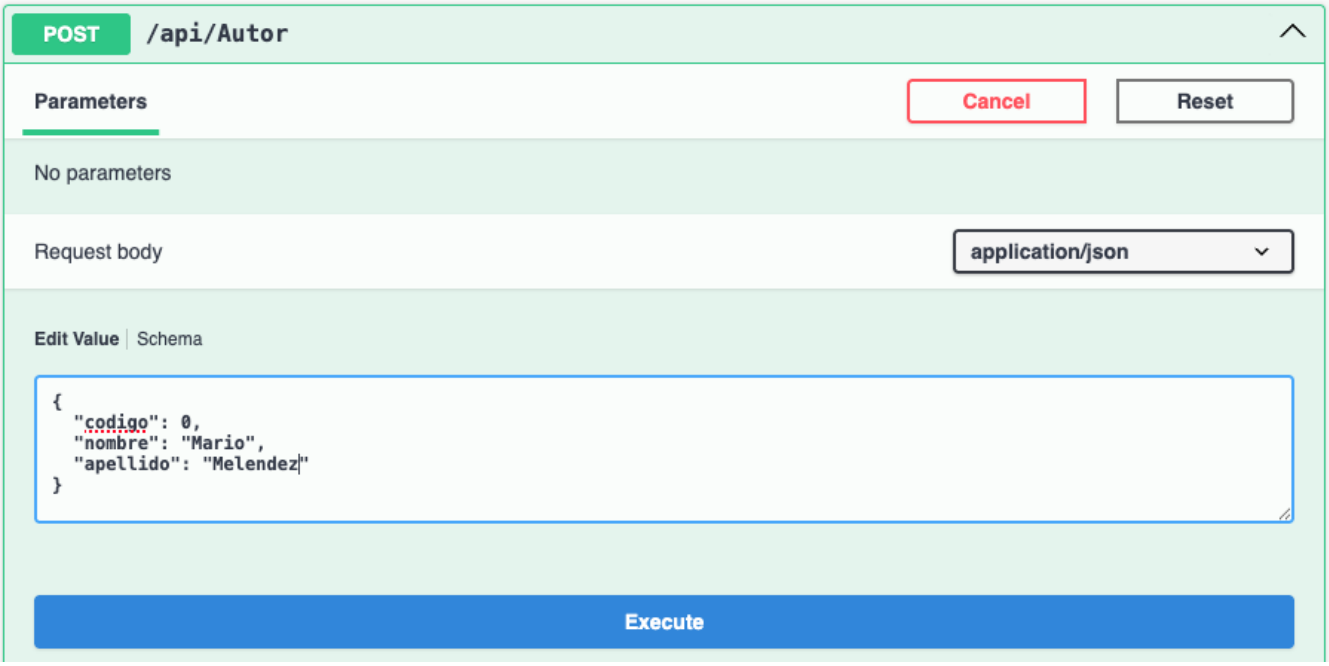
6. Ejecute la aplicación y compruebe el funcionamiento de los endpoints.



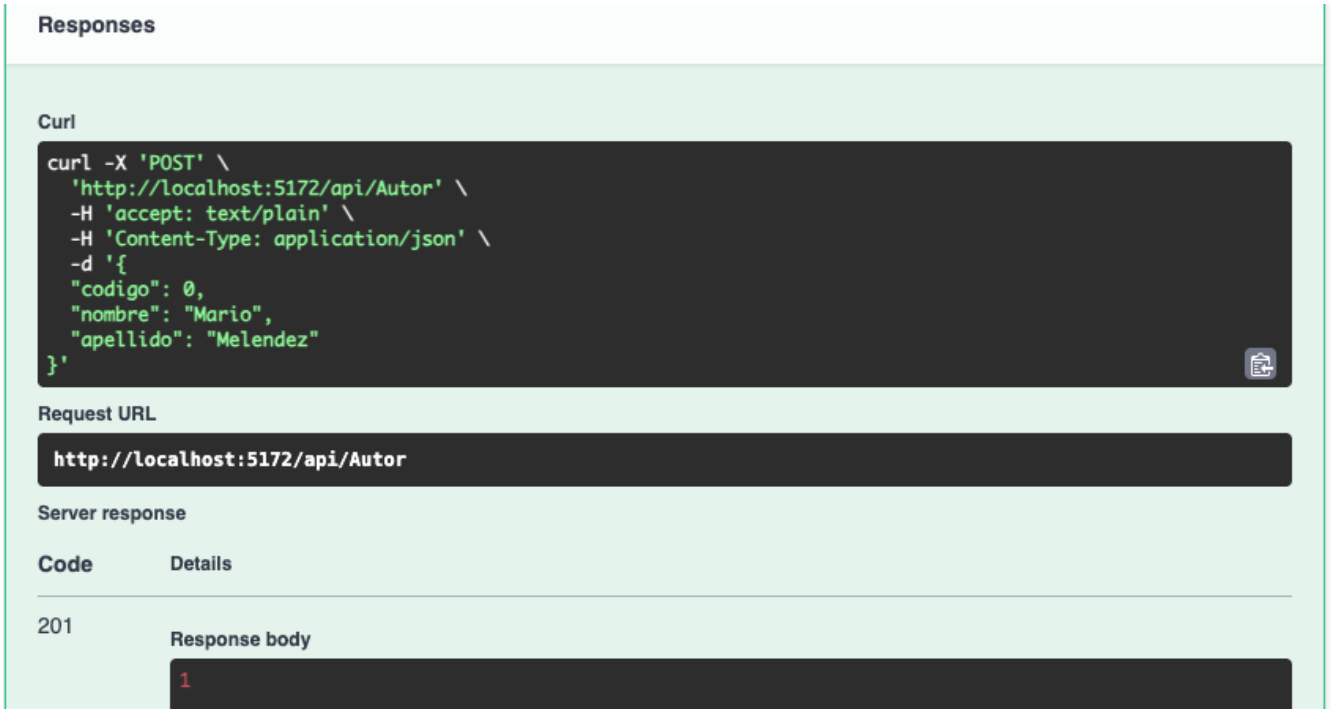
7. Haga clic sobre el **endpoint** que desea probar y luego seleccione la opción **Try it out**.



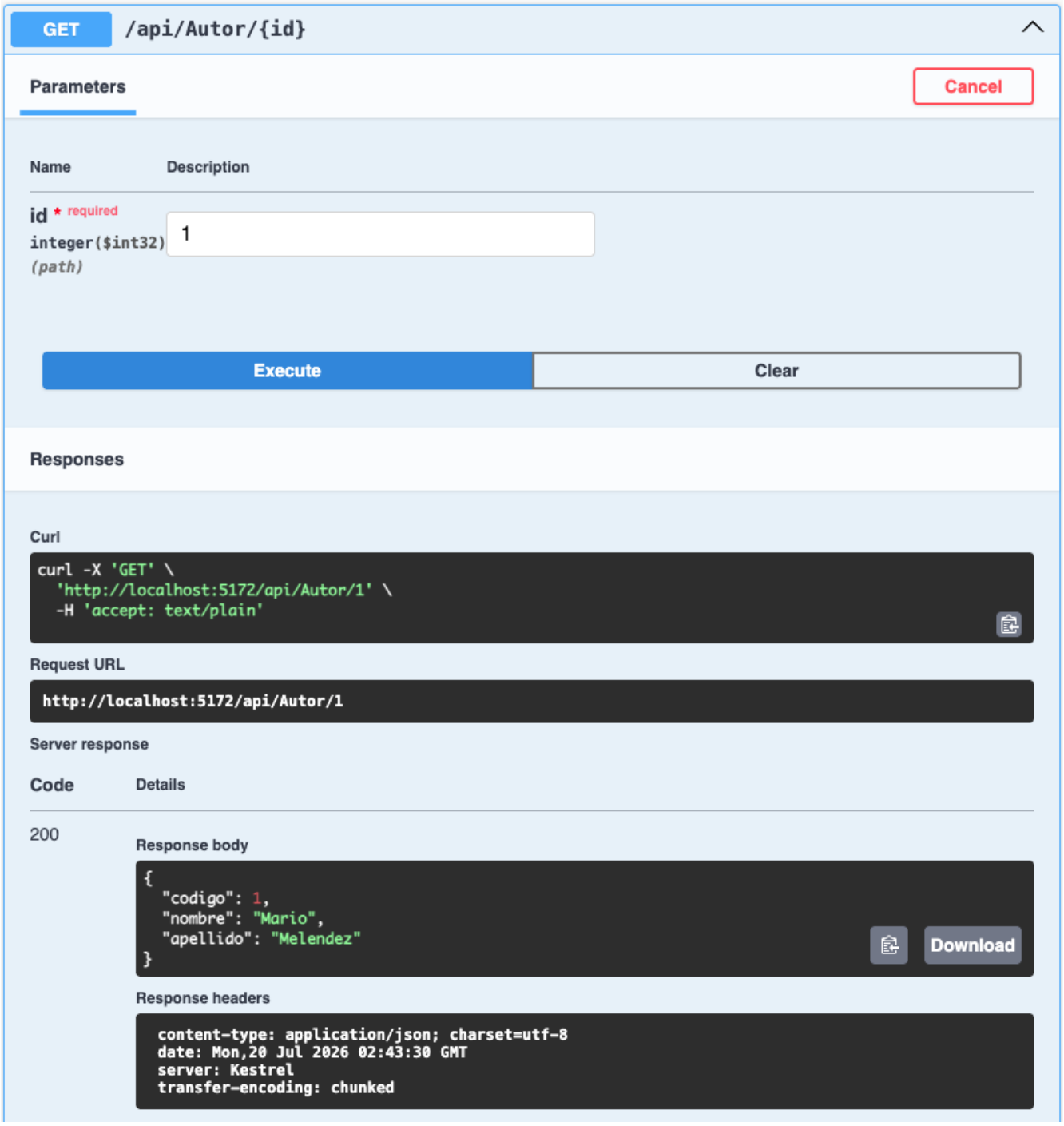
8. Para el caso del método POST modifique el **request body** y haga clic en **Execute**.



9. Si el registro fue insertado exitosamente, en la sección Responses podrá observar el ID del elemento insertado según la definición del contrato de la API.



10. Compruebe el registro insertado haciendo uso del endpoint **/api/Autor/{id}** colocando en el campo **id** el identificador del registro insertado, posteriormente haga clic en **Execute**.



11. Realice pruebas con el resto de los **endpoint** y compruebe el resultado.

IV. DESARROLLO DE HABILIDADES

Ejercicio 1

Implemente en la API la funcionalidad para los modelos Editoriales y Libros siguiendo la arquitectura N-Capas propuesta en la guía.

V. BIBLOGRAFÍA

- BRADFORD, M.; VALVERDE, R. & TALLA, M. (2010). Modern ERP: Select, Implement & Use Todays Advanced Business Systems. North Carolina, State University: Universidad de Carolina del Norte.
- VALVERDE, R. (2012). Information Systems Reengineering for Modern Business Systems. IGI Global, USA.
- SHIELDS, M. (2001). E-Business and ERP Rapid Implementation and Project.